FitFlow

Personal Fitness Dashboard

Software Requirements Specification (SRS)

Student Name	Harsh Kumar
Roll No.	22515000053
Branch	B.Tech - Computer Science & Engineering (CSE)
Section	NC
Document Version	1.0
Prepared For	Frontend Engineering Course Evaluation
Project Category	Responsive Frontend Web Application
Standard Followed	IEEE 830 / IEEE 29148 SRS Format
Date	18 August 2026
Status	Final Draft — For Academic Submission

This document defines the functional and non-functional requirements, system architecture, data model, and constraints for the FitFlow personal fitness dashboard project, prepared for academic development and evaluation purposes.

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **FitFlow**, a responsive, frontend-driven personal fitness dashboard. The purpose of this document is to provide a clear, complete, and unambiguous description of the system's intended behaviour so that it can be used as the primary reference for design, development, testing, and evaluation of the project by students and faculty.

1.2 Document Conventions

Requirements are labelled using the format **FR-[Module]-[Number]** for functional requirements and **NFR-[Number]** for non-functional requirements. Priority levels used are **High**, **Medium**, and **Low**. All mandatory requirements are expressed using the word "shall"; desirable but optional requirements use "should".

1.3 Intended Audience

- Student development team (designers and developers of FitFlow)
- Course faculty / project evaluators / examiners
- Peer reviewers assessing project documentation
- Future contributors extending FitFlow into backend-integrated phases

1.4 Project Scope

FitFlow is a **single-user-focused, browser-based fitness dashboard** that allows an individual to plan and log workouts, track daily water intake and sleep hours, manage simple meal plans, set fitness goals, and visualize personal progress through interactive charts. The current phase of the project is implemented as a **responsive frontend web application** using browser-based storage for data persistence, with the system architected so that a backend/database layer (Section 7.2) can be integrated in a future phase without major redesign.

The scope is intentionally kept realistic for a college-level project team: it does not include social/multi-user features, hardware wearable integration, real-time chat, or payment processing, as these are outside the intended academic scope.

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
FR	Functional Requirement
NFR	Non-Functional Requirement
SPA	Single Page Application
API	Application Programming Interface

Term	Definition
CRUD	Create, Read, Update, Delete
Local Storage	Browser-based key-value storage used for client-side persistence
Responsive Design	UI layout that adapts to different screen sizes (mobile, tablet, desktop)

1.6 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- IEEE/ISO/IEC 29148:2018 — Systems and Software Engineering — Requirements Engineering
- W3C Web Content Accessibility Guidelines (WCAG) 2.1
- Material Design / Human Interface Guidelines — for responsive UI reference

2. Overall Description

2.1 Product Perspective

FitFlow is a **new, standalone product**. It is not a modification of an existing system. It is designed as a self-contained frontend application that can operate independently using local browser storage, and is architected to later connect to a REST API and database backend for multi-device synchronization. The system follows a modular, component-based design so individual modules (workout, water, sleep, meals, goals) can be developed, tested, and evaluated independently.

2.2 Product Functions Summary

At a high level, FitFlow allows a user to:

- Create and manage a personal profile with basic fitness details
- Plan, schedule, and log workouts from customizable routines
- Track daily water intake against a personalized hydration goal
- Log sleep duration and quality, and view sleep trends
- Create and manage simple daily/weekly meal plans with calorie estimates
- Set measurable fitness goals (e.g., weight target, workout frequency) and monitor progress
- Visualize progress across all modules using interactive charts and summary cards
- Customize the dashboard layout and appearance (e.g., themes, widget order)

2.3 User Classes and Characteristics

User Class	Description	Frequency of Use
Primary User (Fitness Enthusiast)	General end user who logs daily fitness activity, sets goals, and views progress. No technical expertise required.	High
Guest / New User	First-time visitor exploring the dashboard before creating a full profile.	Medium
Administrator (Project Evaluator)	Faculty/evaluator reviewing the working system and documentation; interacts as a standard user for demonstration.	Low (project context only)

2.4 Operating Environment

- **Client Side:** Modern web browsers — Google Chrome, Mozilla Firefox, Microsoft Edge, Safari (latest two major versions)
- **Devices:** Desktop, laptop, tablet, and mobile phones (responsive breakpoints for all)
- **Operating Systems:** Any OS capable of running a modern browser (Windows, macOS, Linux, Android, iOS)
- **Hosting (Development/Demo):** Static hosting platforms such as Netlify, Vercel, or GitHub Pages

2.5 Design and Implementation Constraints

- The current phase must function fully as a **frontend-only application**, using local/browser storage; no server-side dependency is mandatory for core functionality
- The system shall be built using standard, freely available, open-source web technologies suitable for a student project timeline
- The UI shall follow a consistent red-and-neutral colour theme (see Section 4.1) and shall be fully responsive
- Development timeline is constrained to a single academic semester; scope is limited accordingly
- The system shall not require any paid third-party service to demonstrate core functionality

2.6 Assumptions and Dependencies

- Users have access to a device with a modern, JavaScript-enabled web browser and internet access to initially load the application
- Users are responsible for the accuracy of manually entered health data (e.g., meals, sleep hours)
- The application is a wellness planning tool and is **not a certified medical device**; it does not provide medical advice
- Future backend integration (Section 7.2) is assumed to use a REST API with a relational or document database

3. System Features (Functional Requirements)

This section describes each major module of FitFlow along with its associated functional requirements, priority, and expected system behaviour.

3.1 User Authentication & Profile Management

Allows a user to create and manage a personal account/profile that stores basic fitness parameters used to personalize the dashboard.

Req. ID	Requirement Description	Priority
FR-AUTH-01	The system shall allow a new user to register using name, email, and password.	High
FR-AUTH-02	The system shall allow a registered user to log in and log out securely.	High
FR-AUTH-03	The system shall allow the user to create/edit a profile with age, gender, height, weight, and activity level.	High
FR-AUTH-04	The system shall calculate and display basic derived metrics (e.g., BMI) from profile data.	Medium
FR-AUTH-05	The system shall persist user session data locally so the user does not need to log in on every visit.	Medium

3.2 Workout Planner Module

Enables users to plan, schedule, and track workout routines and individual exercises.

Req. ID	Requirement Description	Priority
FR-WRK-01	The system shall allow the user to create a workout plan by selecting exercises from a predefined category list (e.g., cardio, strength, flexibility).	High
FR-WRK-02	The system shall allow the user to schedule workouts on specific days of the week.	High
FR-WRK-03	The system shall allow the user to mark a scheduled workout as completed.	High
FR-WRK-04	The system shall allow the user to edit or delete an existing workout plan.	Medium
FR-WRK-05	The system shall display a weekly workout calendar/list view.	Medium
FR-WRK-06	The system shall track workout streaks (consecutive days/weeks with completed workouts).	Low

3.3 Water Intake Tracker

Allows users to log daily water consumption and track progress toward a hydration goal.

Req. ID	Requirement Description	Priority
FR-WTR-01	The system shall allow the user to set a daily water intake goal (in ml or litres).	High

Req. ID	Requirement Description	Priority
FR-WTR-02	The system shall allow the user to log water intake in customizable increments (e.g., glass, bottle, custom ml).	High
FR-WTR-03	The system shall display a visual progress indicator (e.g., progress bar/ring) showing intake vs. goal for the current day.	High
FR-WTR-04	The system shall reset the daily water log automatically at the start of a new day.	Medium
FR-WTR-05	The system shall display a historical view (weekly) of water intake trends.	Medium

3.4 Sleep Tracker

Allows users to log sleep duration and quality, and view trends over time.

Req. ID	Requirement Description	Priority
FR-SLP-01	The system shall allow the user to manually log sleep start time and wake time.	High
FR-SLP-02	The system shall automatically calculate total sleep duration from logged times.	High
FR-SLP-03	The system shall allow the user to rate sleep quality (e.g., Poor/Average/Good).	Medium
FR-SLP-04	The system shall display a weekly sleep trend chart comparing duration against a recommended sleep goal.	Medium
FR-SLP-05	The system shall allow the user to edit or delete a previously logged sleep entry.	Low

3.5 Meal Plan Manager

Allows users to plan simple daily/weekly meals and track estimated calorie intake.

Req. ID	Requirement Description	Priority
FR-MEAL-01	The system shall allow the user to add meals under categories: Breakfast, Lunch, Dinner, and Snacks.	High
FR-MEAL-02	The system shall allow the user to select food items from a predefined list with associated approximate calorie values.	High
FR-MEAL-03	The system shall calculate and display total daily calorie intake based on logged meals.	High
FR-MEAL-04	The system shall allow the user to set a daily calorie target and compare it against actual intake.	Medium
FR-MEAL-05	The system shall allow the user to create and save a reusable weekly meal plan template.	Low

3.6 Goal Setting Module

Allows users to define measurable, trackable fitness goals across modules.

Req. ID	Requirement Description	Priority
FR-GOAL-01	The system shall allow the user to create a fitness goal with a target value, unit, and target date (e.g., target weight, workout frequency).	High
FR-GOAL-02	The system shall automatically update goal progress based on related logged data (e.g., workouts completed).	High
FR-GOAL-03	The system shall display goal progress as a percentage and visual indicator.	High
FR-GOAL-04	The system shall notify (via in-app message) the user when a goal is achieved.	Medium
FR-GOAL-05	The system shall allow the user to edit or archive completed/expired goals.	Low

3.7 Progress Visualization Dashboard

The central dashboard that aggregates data from all modules into an interactive, customizable view.

Req. ID	Requirement Description	Priority
FR-DASH-01	The system shall display a summary dashboard with widgets/cards for workouts, water, sleep, meals, and goals.	High
FR-DASH-02	The system shall render interactive charts (line, bar, or radial) for weekly/monthly progress per module.	High
FR-DASH-03	The system shall allow the user to reorder or hide dashboard widgets according to preference.	Medium
FR-DASH-04	The system shall update dashboard visuals in real time as new data is logged, without requiring a page reload.	High

3.8 Settings & Customization

Allows users to personalize the application experience.

Req. ID	Requirement Description	Priority
FR-SET-01	The system shall allow the user to switch between light and dark visual themes.	Medium
FR-SET-02	The system shall allow the user to set measurement units (metric/imperial).	Medium
FR-SET-03	The system shall allow the user to export their logged data as a downloadable file (e.g., CSV/JSON).	Low
FR-SET-04	The system shall allow the user to reset/clear all stored data with a confirmation prompt.	Medium

4. External Interface Requirements

4.1 User Interfaces

- The UI shall follow a clean, card-based dashboard layout with a primary red-and-white theme, using red as the accent/brand colour for headers, buttons, active states, and charts.
- The UI shall be fully responsive across mobile (<600px), tablet (600–1024px), and desktop (>1024px) breakpoints.
- Navigation shall be provided via a sidebar (desktop) collapsing into a bottom/hamburger navigation on mobile.
- All interactive elements (buttons, forms, sliders) shall provide visible hover/active/focus states.
- Charts and progress indicators shall use accessible color contrast and include textual labels, not colour alone.

4.2 Hardware Interfaces

FitFlow does not require any specialized hardware. It shall run on any standard computing device (desktop, laptop, tablet, smartphone) equipped with a display, input device (keyboard/mouse/touchscreen), and network connectivity for initial load.

4.3 Software Interfaces

Interface	Purpose
Web Browser	Renders the FitFlow single-page application (HTML/CSS/JavaScript)
Browser Local Storage / IndexedDB	Client-side persistence of user data in current phase
Charting Library	Renders interactive charts for progress visualization
Future REST API (Phase 2)	Will handle authentication, data sync, and multi-device persistence

4.4 Communication Interfaces

In the current frontend-only phase, no external network communication is required for core functionality beyond the initial application load. In the future backend-integrated phase, the system shall communicate with a REST API over HTTPS using JSON payloads.

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The dashboard shall load within 3 seconds on a standard broadband connection.	High
NFR-02	Chart re-rendering after data entry shall complete within 1 second.	Medium

5.2 Security Requirements

ID	Requirement	Priority
NFR-03	User passwords (future backend phase) shall be stored using a secure hashing algorithm; never in plain text.	High
NFR-04	All locally stored data shall remain isolated per browser profile and shall not be accessible to other sites.	Medium
NFR-05	Future API communication shall be encrypted using HTTPS/TLS.	High

5.3 Usability Requirements

ID	Requirement	Priority
NFR-06	The interface shall follow a consistent design language across all modules (typography, spacing, colour).	High
NFR-07	The system shall provide clear error messages and input validation feedback.	High
NFR-08	New users shall be able to log their first workout, water entry, or meal within 2 minutes of first use without external help.	Medium

5.4 Reliability & Availability

ID	Requirement	Priority
NFR-09	The system shall not lose previously logged data on browser refresh or accidental tab close.	High
NFR-10	The system shall handle invalid or empty form submissions gracefully without crashing.	High

5.5 Maintainability & Portability

- The codebase shall follow a modular, component-based structure to allow independent updates to each module.
- The system shall be portable across major modern browsers without requiring browser-specific code.
- Configuration (e.g., theme colours, calorie/goal defaults) shall be centralized to simplify future changes.

6. System Architecture

6.1 Architectural Overview

FitFlow follows a **modular, component-based frontend architecture**. The application is organized into independent feature modules (Workout, Water, Sleep, Meals, Goals, Dashboard, Settings) that share a common state layer and design system. Each module reads and writes data through a lightweight data-access layer, which in the current phase talks to browser local storage, and in a future phase can be swapped to call a REST API without changing the module UI code.

Layer	Responsibility
Presentation Layer	Responsive UI components, pages, charts, and navigation
State Management Layer	Manages application state (user profile, logs, goals) shared across modules
Data Access Layer	Abstraction for reading/writing data — local storage today, API calls in future
Persistence Layer (Current)	Browser Local Storage / IndexedDB for offline-capable data persistence
Persistence Layer (Future)	Backend REST API + relational/document database (Section 7.2)

6.2 Technology Stack

Layer/Aspect	Proposed Technology
Frontend Framework	React.js (or equivalent component-based framework)
Styling	CSS3 / Tailwind CSS with a custom red-themed design system
Charting	Chart.js / Recharts for interactive data visualization
State Management	React Context API / lightweight state library
Data Persistence (Phase 1)	Browser Local Storage / IndexedDB
Version Control	Git & GitHub
Future Backend (Phase 2)	Node.js + Express REST API with MongoDB or MySQL

6.3 Component Diagram (Description)

At a conceptual level, the application root renders a persistent **Navigation Component** and a **Dashboard Container**. The Dashboard Container composes independent widget components: *WorkoutWidget*, *WaterWidget*, *SleepWidget*, *MealWidget*, and *GoalWidget* — each of which consumes data from the shared State Management Layer and renders its own chart/progress view. A *SettingsPanel* component controls theme and preference state consumed globally. This decoupled structure allows each module to be built and tested independently, which is well suited to parallel development among student team members.

7. Data Requirements

7.1 Conceptual Data Model

The core entities managed by FitFlow and their relationships are summarized below. This conceptual model applies to both the current local-storage implementation and the proposed future database schema.

Entity	Description
User	Represents the individual using the application; stores profile and preference data.
WorkoutPlan	A user-created plan containing one or more scheduled Exercises.
Exercise	An individual exercise entry (name, category, duration/reps, completion status).
WaterLog	A record of water intake for a specific date/time.
SleepLog	A record of sleep start/end time and quality rating for a specific date.
Meal	A logged food item under a meal category with associated calorie value.
Goal	A user-defined target with type, target value, current progress, and deadline.

7.2 Proposed Database Schema (Future Phase)

The following relational schema is proposed for the future backend-integrated phase. Primary keys are denoted PK and foreign keys FK.

Table	Key Fields
users	user_id (PK), name, email, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at
workout_plans	plan_id (PK), user_id (FK), title, scheduled_day, created_at
exercises	exercise_id (PK), plan_id (FK), name, category, duration_min, reps, is_completed
water_logs	log_id (PK), user_id (FK), amount_ml, logged_at
sleep_logs	log_id (PK), user_id (FK), sleep_start, wake_time, quality_rating, logged_date
meals	meal_id (PK), user_id (FK), category, food_item, calories, logged_date
goals	goal_id (PK), user_id (FK), goal_type, target_value, current_value, unit, target_date, status

7.3 Local Storage Strategy (Current Phase)

In the current frontend-only phase, each entity above is represented as a JSON structure persisted under a namespaced key in browser local storage (e.g., `fitflow_user`, `fitflow_workouts`, `fitflow_water_logs`). This mirrors the future database schema so that migration to a real backend requires minimal restructuring of the data-access layer only.

8. Use Case Model

8.1 Actors

- **User** — the primary actor who registers, logs data, sets goals, and views progress.
- **System** — FitFlow application, which processes input and updates visualizations automatically.

8.2 Use Case List

Use Case ID	Use Case Name	Primary Actor
UC-01	Register / Log In	User
UC-02	Update Profile	User
UC-03	Create Workout Plan	User
UC-04	Log Completed Workout	User
UC-05	Log Water Intake	User
UC-06	Log Sleep Entry	User
UC-07	Add Meal to Daily Log	User
UC-08	Set Fitness Goal	User
UC-09	View Progress Dashboard	User
UC-10	Customize Dashboard/Theme	User
UC-11	Export Data	User

8.3 Sample Use Case Descriptions

Use Case UC-04: Log Completed Workout

Field	Description
Preconditions	User is logged in and has at least one scheduled workout plan.
Main Flow	1. User navigates to Workout module. 2. User selects a scheduled exercise. 3. User marks the exercise as completed. 4. System updates workout streak and dashboard progress in real time.
Alternate Flow	If no workout is scheduled for the day, the system prompts the user to create one.
Postconditions	The exercise status is updated and reflected in the Progress Visualization Dashboard.

Use Case UC-08: Set Fitness Goal

Field	Description
Preconditions	User is logged in.

Field	Description
Main Flow	1. User navigates to Goals module. 2. User selects goal type (e.g., target weight, weekly workout count). 3. User enters target value and target date. 4. System saves the goal and begins tracking progress from related logs.
Alternate Flow	If the target date is earlier than today, the system displays a validation error.
Postconditions	A new goal appears in the Goals list and on the dashboard with 0% initial progress.

9. System Models

9.1 Data Flow Diagram (Level 0 & 1 — Description)

Level 0 (Context Diagram): The User interacts with the single process "FitFlow System", providing inputs (profile data, workout logs, water/sleep entries, meal entries, goal definitions) and receiving outputs (dashboard visuals, progress summaries, notifications).

Level 1 (Decomposition): The FitFlow System process decomposes into five core sub-processes — *Manage Profile*, *Manage Workouts*, *Manage Health Logs* (water & sleep), *Manage Meals*, and *Manage Goals* — each of which reads from and writes to its corresponding data store, and feeds aggregated data into the *Generate Dashboard View* process for visualization.

9.2 State Diagram — Workout Session

A scheduled exercise instance moves through the following states: **Scheduled** → **In Progress** → **Completed**, with an alternate transition from **Scheduled** → **Skipped** if the user does not perform it by end of day. Completed and Skipped are terminal states for that instance and contribute to streak and goal calculations respectively.

10. Software Testing Requirements

10.1 Testing Approach

Testing shall be conducted at unit, integration, and system levels to ensure each module functions correctly on its own and in combination with others, and that the application behaves correctly across supported devices and browsers.

- **Unit Testing:** Individual components (e.g., calorie calculator, streak counter, BMI calculator) tested in isolation.
- **Integration Testing:** Verifying data flow between modules and the shared state/dashboard layer.
- **System Testing:** End-to-end validation of complete user workflows (e.g., log meal → dashboard updates).
- **UI/Responsiveness Testing:** Verifying layout correctness across mobile, tablet, and desktop breakpoints.
- **User Acceptance Testing (UAT):** Faculty/peer walkthrough of core use cases prior to final submission.

10.2 Test Case Categories

Category	Coverage
TC-AUTH	Registration, login, logout, and profile validation scenarios.
TC-WRK	Creating, scheduling, completing, editing, and deleting workouts.
TC-WTR	Logging water intake, goal comparison, and daily reset behaviour.
TC-SLP	Logging sleep entries and validating duration calculations.
TC-MEAL	Adding meals, calorie totals, and daily calorie goal comparisons.
TC-GOAL	Goal creation, automatic progress updates, and completion notifications.
TC-DASH	Dashboard widget rendering, chart accuracy, and real-time updates.
TC-UI	Responsive layout, theme switching, and accessibility checks.

11. Appendices

11.1 Glossary

Term	Definition
Widget	A self-contained dashboard component displaying a specific module's summary/chart.
Streak	The number of consecutive days/weeks a user has met a specific activity target.
Goal Progress	The percentage completion of a user-defined goal, calculated from related logs.
Breakpoint	A defined screen-width threshold at which the responsive layout changes.

11.2 Future Scope

- Integration with a full backend (Node.js/Express + database) as outlined in Section 7.2
- Wearable device / smartwatch data integration (steps, heart rate)
- Social features such as friend challenges or leaderboards
- AI-based personalized workout and meal recommendations
- Push notifications and reminders for hydration, sleep, and workouts
- Multi-language support

11.3 Requirement Traceability Matrix (Sample)

Requirement ID	Use Case	Design Component	Test Category
FR-WRK-01	UC-03	Workout Planner UI	TC-WRK
FR-WTR-02	UC-05	Water Tracker UI	TC-WTR
FR-SLP-01	UC-06	Sleep Tracker UI	TC-SLP
FR-MEAL-03	UC-07	Meal Manager UI	TC-MEAL
FR-GOAL-02	UC-08	Goal Module UI	TC-GOAL
FR-DASH-02	UC-09	Dashboard Charts	TC-DASH

— End of Document —